# CHAPTER ONE

# Compiler Design

**Chapter Outline**

- What is compiler Design?
- Language Processing System
- Types of Compiler Design
- Phases of Compiler Design
- Compiler construction tools

## 1.1. Introduction

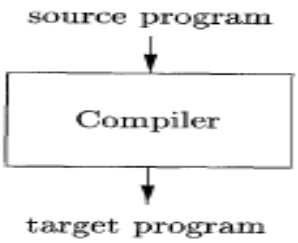
Computers are a balanced mix of software and hardware. Hardware is just a piece of mechanical device and a compatible software is controlling its functions. Hardware understands instructions in the form of electronic charge, which is the counterpart of binary language in software programming. Binary language has only two alphabets, 0 and 1. To instruct, the hardware codes must be written in binary format, which is simply a series of 1s and 0s. It would be a difficult and cumbersome task for computer programmers to write such codes, which is why we have compilers to write such codes.

In order to reduce the complexity of designing and building computers, nearly all of these are made to execute relatively simple commandes (but do So very quickly). A program for a computer must be built by combining these very simple commands into a program in which is *called* ***machine language***. Most programming is done using a *high-level programming language* -> But this language can be very different from the machine language that the computer can execute.

===>This is where the **compiler** comes in

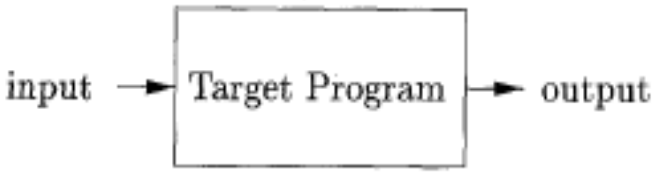
***What's a compiler?***

A compiler translates a program written in a high-level programming language that is suitable for human programmers into the low-level machine language that is required by computers and spots and reports obvious programmer mistakes.



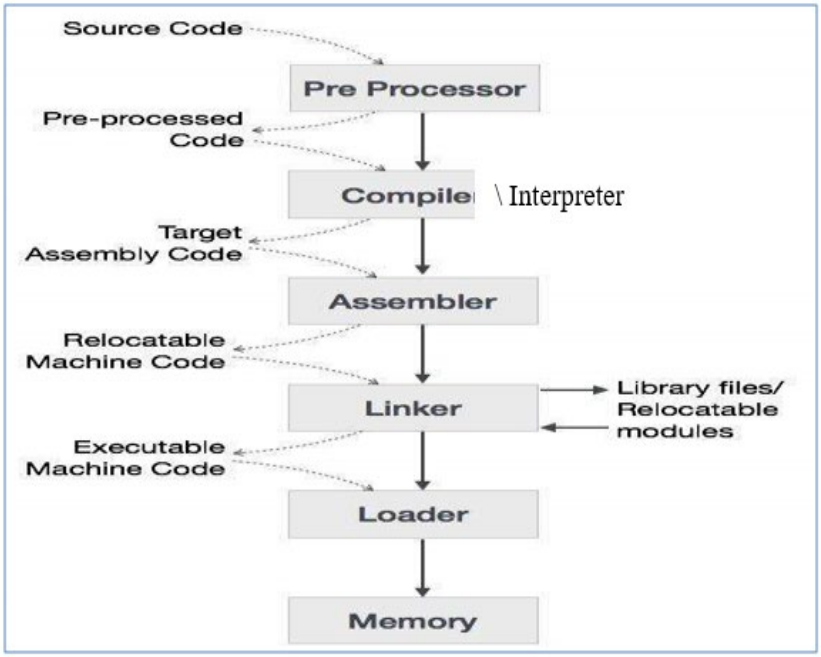
**Running the target program**

- If the target program is an executable machine-language program, it can then be called by the user to process inputs and produce outputs.




## Language Processing System

We have learnt that any computer system is made of hardware and software. The hardware understands a language, which humans cannot understand. Therefore, we write programs in high-level language, which is easier for us to understand and remember. These programs are then fed into a series of tools and OS components to get the desired code that can be used by the machine. This is known as Language Processing System. Figure-1 below.




The high-level language is converted into binary language in various phases. A compiler is a program that converts high-level language to assembly language. Similarly, an assembler is a program that converts the assembly language to machine-level language.

## Preprocessor

A preprocessor, generally considered as a part of compiler, is a tool that produces input for compilers. It deals with macro-processing, augmentation; file **inclusion**, language extension, etc.

**Compiler**

This is a program that reads a high level programming language(source code) and produces an equivalent low level target machine executable code which can be run against various inputs. A compiler operates in logically interrelated phases whereby source code is translated from one representation and produces output in another representation.

**Interpreter (as Compiler)**

An interpreter, like a compiler, translates high-level language into low-level machine language. The difference lies in the way they read the source code or input. A compiler reads the whole source code at once, creates tokens, checks semantics, generates intermediate code, executes the whole program and may involve many passes. In contrast, an interpreter reads a statement from the input converts it to an intermediate code, executes it, then takes the next statement in sequence. If an error occurs, an interpreter stops execution and reports it. Whereas a compiler reads the whole program even if, it encounters several errors.

**Assembler**

An assembler translates assembly language programs into machine code. The output of an assembler is called an object file, which contains a combination of machine instructions as well as the data required to place these instructions in memory.

**Linker**

Linker is a computer program that links and merges various object files together in order to make an **executable file**. All these files might have been compiled by separate assemblers. The major task of a linker is to search and locate referenced module/routines in a program and to determine the memory location where these codes will be loaded, making the program instruction to have absolute references.

**Loader**

Loader is a part of operating system and is responsible for loading executable files into memory and execute them. It calculates the size of a program (instructions and data) and creates memory space for it. It initializes various registers to initiate execution.

## Types of Compiler

- **Native-compiler**

  Native compiler are compilers that generates code for the same Platform on which it runs. It converts high language into computer's native language. For example Turbo C or GCC compiler

- **Cross-compiler**

  A compiler that runs on platform (A) and is capable of generating executable code for platform (B) is called a cross-compiler.

- **Source-to-source Compiler/transcompiler**

  A compiler that takes the source code of one programming language and translates it into the source code of another programming language is called a source-to-source compiler.

- **Decompiler**

  Basically, it is not a compiler. It is just the reverse of the compiler. It converts the machine code into high-level language

**Features of a Compiler**

- Compilation speed.
- The correctness of machine code.
- The meaning of code should not change.
- Speed of machine code.
- Good error detection.
- Checking the code correctly according to grammar.

**Uses/Application of Compilers**

- Helps to make the code independent of the platform.
- Makes the code free of syntax and semantic errors.
- Generate executable files of code.
- Translates the code from one language to another.

**Compiler Design - Architecture**

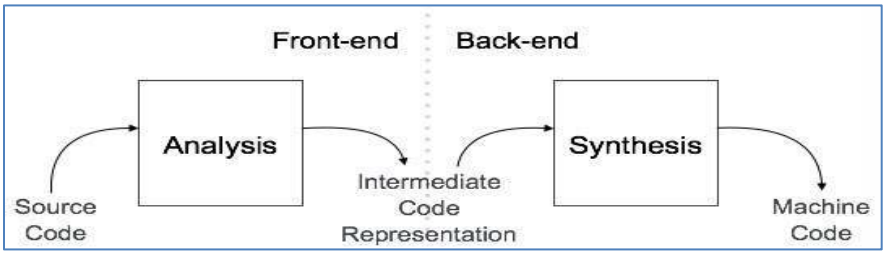
A compiler can broadly be divided into two phases based on the way they compile.

### 1. Analysis Phase

Known as the front-end of the compiler, the analysis phase of the compiler reads the source program, divides it into core parts and then checks for lexical, grammar and syntax errors. The analysis phase generates an intermediate representation of the source program and symbol table, which should be fed to the Synthesis phase as input.

## 2. Synthesis Phase

Known as the back-end of the compiler, the synthesis phase generates the target program with the help of intermediate source code representation and symbol table.

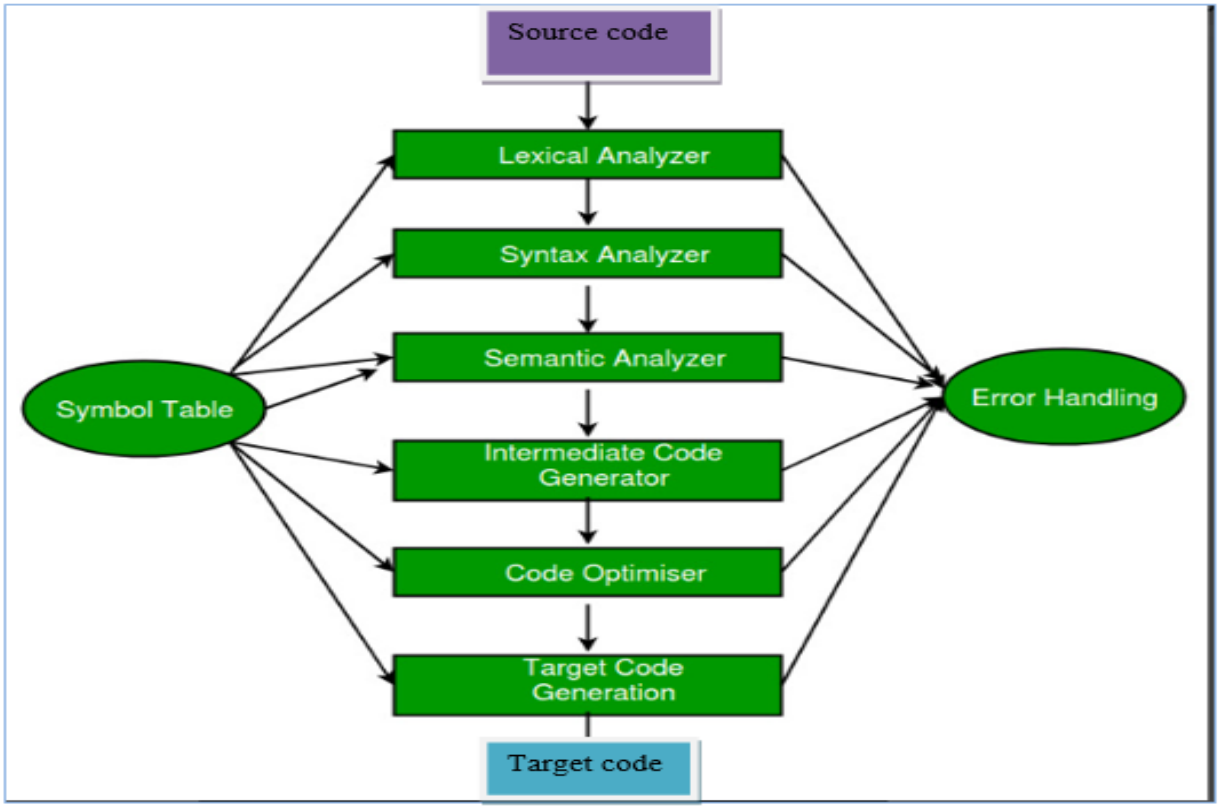



*A compiler can have many phases and passes.*

- **Pass**: A pass refers to the traversal of a compiler through the entire program.
- **Phase**: A phase of a compiler is a distinguishable stage, which takes input from the previous stage, processes and yields output that can be used as input for the next stage. A pass can have more than one phase.

**Phases of Compiler**

The compilation process is a sequence of various phases. Each phase takes input from its previous stage, has its own representation of source program, and feeds its output to the next phase of the compiler. Let us understand the phases of a compiler from the following figure.



**Analysis Phase are:** Lexical Analysis, Syntax Analysis, Semantic analysis

**Synthesis Phase are:** Code Optimization, Code Generation,

### 1. Lexical Analysis

The first phase compiler can be said to a scanner, which works as a text scanner. This phase scans the source code as a stream of characters and converts it into meaningful **lexemes**.

### 2. Syntax Analysis

The next phase is called the syntax analysis or **parsing**. It takes the token produced by lexical analysis as input and generates a parse tree or syntax tree.

### 3. Semantic Analysis

Semantic analysis checks whether the parse tree constructed follows the rules of language. For example, assignment of values is between compatible data types, and adding string to an integer. Also, the semantic analyzer keeps track of identifiers, their types and expressions; whether identifiers are declared before use or not etc.

### 4. Intermediate Code Generation

After semantic analysis the compiler generates an intermediate code of the source code for the target machine. It represents a program for some abstract machine. It is in between the high-level language and the machine language. This intermediate code should be generated in such a way that it makes it easier to be translated into the target machine code.

### 5. Code Optimization

The next phase does code optimization of the intermediate code. Optimization can be assumed as something that removes unnecessary code lines, and arranges the sequence of statements in order to speed up the program execution without wasting resources (CPU, memory).

### 6. Code Generation

In this phase, the code generator takes the optimized representation of the intermediate code and maps it to the target machine language. The code generator translates the intermediate code into a sequence of (generally) re-locatable machine code.

### Symbol Table

It is a data-structure maintained throughout all the phases of a compiler. All the identifier's names along with their types are stored here. The symbol table makes it easier for the compiler to quickly search the identifier record and retrieve it.

### Error Handling

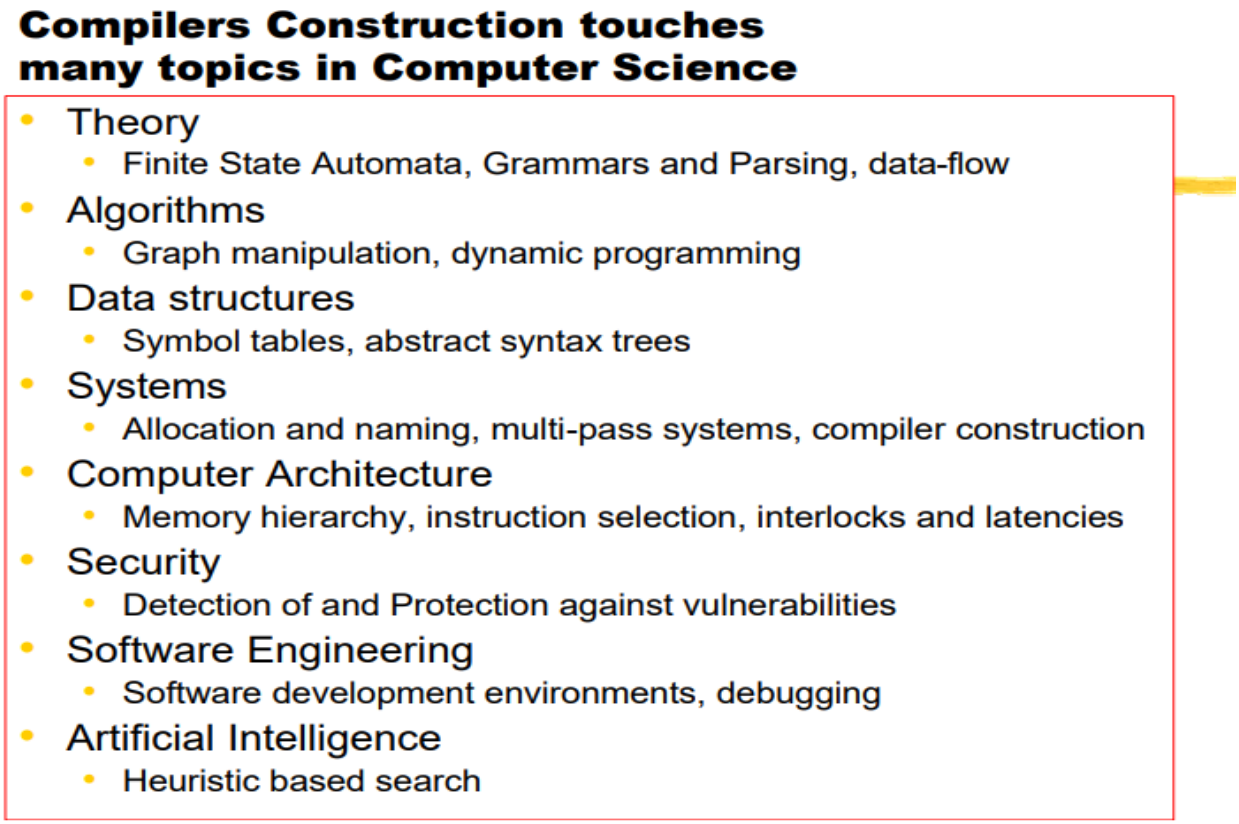
Detection and reporting of errors in the source program is the main function of the compiler. An error can occur at any phase of compilation. A good compiler must determine the line number of the program exactly, where the errors have occurred. Various errors that can occur at a different level of compilation.

❖ **How to translate?**

The high level languages and machine languages differ in level of abstraction. At machine level we deal with memory locations, registers whereas these resources are never accessed in high level languages. But the level of abstraction differs from language to language.

In order to translate a high level code to a machine code one needs to go step by step, with each step doing a particular task and passing out its output for the next step in the form of another program representation. The steps can be parse tree generation, high level intermediate code generation, low level intermediate code generation, and then the machine language conversion. As the translation proceeds the representation becomes more and more machine specific, increasingly dealing with registers, memory locations etc.

- Translate in steps. Each step handles a reasonably simple, logical, and well defined task
- Design a series of program representations
- Intermediate representations should be amenable to program manipulation of various kinds (type checking, optimization, code generation etc.)
- Representations become more machine specific and less language specific as the translation proceeds



## Compiler construction tools

Writing a compiler is tedious and time consuming task. There are some specialized tools for helping in implementation of various phases of compilers. These tools are called compiler construction tools. These tools are also called as compilercompiler, compiler-generators, or translator writing system. Various compiler construction tools are given as below.

1. **Scanner Generator** : These generators generate Lexical analyzers. The specification given to these generators are in the form of regular expressions. The UNIX has utility for a scanner generator called LEX. The specification given to the LEX consists of regular expressions for representing various tokens.
2. **Parser Generators** : These produce the syntax analyzer. The specification given to these generators is given in the form of context free grammar. Typically UNIX has a tool called YACC which is parser generator.

3. **Syntax-directed translation engines** : In this tool the parse tree is scanned completely to generate an intermediate code. The translation is done for each node of the tree.

4. **Automatic code generator** : These generators take an intermediate code as input and converts each rule of intermediate language into equivalent machine language. The template matching technique is used. The intermediate code statements are replaced templates that represent the corresponding sequence of machine instructions.

5. **Data flow engines** : The data flow analysis is required to perform good code optimization. The data flow engines are basically useful in code optimization

" End"